

Installation et configuration Java

Spring Boot avec Junie

1. Le langage JAVA

Version du langage JAVA

Suivre la version d'un langage (son évolution) :

Pour le JDK 24, allez sur le site d'Oracle pour voir les nouveautés apportées par cette version :

<https://docs.oracle.com/en/java/javase/24/language/java-language-changes.html#GUID-6459681C-6881-45D8-B0DB-395D1BD6DB9B>

Attention, JDK 24 n'est pas une version LTS (support long terme). Utiliser de préférence une version dont la maintenance sera assurée pendant plusieurs années pour vos projets : version 21 ou la toute prochaine [version 25](#).

Évolution du langage JAVA

A chaque version, le langage évolue et s'améliore. Voir à chaque fois la documentation.

Mise en œuvre de Record :

<https://www.developer.com/java/java-record-class/>

Faire fonctionner un programme simple avec Record.

Mise en oeuvre de Switch :

<https://java.tanguyleloch.com/formations/prise-de-decision-java/nouveau-switch-avec-gardes-en-java-21/>

Faire fonctionner un programme simple avec Switch case.

2. Outils de commandes et gestions en ligne

Nous nous intéressons à **curl** et **maven** (qui a été vu l'année précédente).

Installation de curl (permet de nombreuses commandes en ligne) :

Télécharger curl : <https://curl.se/windows/>

Décompresser et installer curl dans un répertoire.

Puis mettre curl dans une variable d'environnement système tel que :

http://support.ptc.com/help/vuforia/studio/fr/index.html#page/Studio_Help_Center/digital_twin/digital_twin_202_download_cURL.html

Remarque : si l'installation prend trop de temps (lenteur du téléchargement par exemple), faites cette installation plus tard, ce n'est pas pénalisant pour la suite.

Exemple de commandes curl : <https://www.malekal.com/16-exemples-utilisation-de-la-commande-curl/>

Installez vos variables d'environnement

par exemple :

CURL_HOME	C:\Program Files\curl-8.14.1
DriverData	C:\Windows\System32\Drivers\DriverData
JAVA_HOME	C:\Program Files\Java\jdk-21
MAVEN_HOME	C:\Program Files\Maven\apache-maven-3.9.10

N'oubliez pas d'ajouter ces répertoires à votre PATH.

Vérifiez vos variables d'environnement

avec la commande : echo %variable%

```
C:\Users\caspar5>echo %JAVA_HOME%  
C:\Program Files\Java\jdk-21
```

3. Le framework Spring Boot

<https://spring.io/projects/spring-boot>

Choisir la bonne version (GA = Généralement Admise) :

OVERVIEW	LEARN	SUPPORT	SAMPLES
Documentation			
Each Spring project has its own; it explains in great details how you can use project features and what you can achieve with them.			
3.5.5	CURRENT GA	Reference Doc.	Api Doc.
4.0.0-M2	PRE	Reference Doc.	Api Doc.
4.0.0-SNAPSHOT	SNAPSHOT	Reference Doc.	Api Doc.
3.5.6-SNAPSHOT	SNAPSHOT	Reference Doc.	Api Doc.
3.4.10-SNAPSHOT	SNAPSHOT	Reference Doc.	Api Doc.
3.4.9	GA	Reference Doc.	Api Doc.
3.3.13	GA	Reference Doc.	Api Doc.
3.2.12	GA	Reference Doc.	Api Doc.

Regarder la [durée du support](#) pour les entreprises :



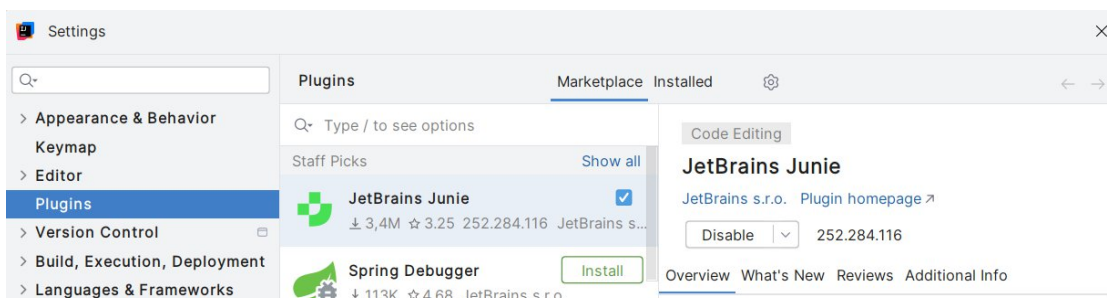
[Spring initializr](#)

Choisir les métadonnées (nommage), les dépendances, la version de Java pour configurer votre projet Spring :

The Spring Initializr configuration interface. It includes sections for Project, Language, Spring Boot, Project Metadata, and Dependencies. The Project section shows Gradle - Groovy selected. The Language section shows Java selected. The Spring Boot section shows 3.5.5 selected. The Project Metadata section includes fields for Group, Artifact, Name, Description, and Package name. The Dependencies section has a button to add dependencies. The Java version section shows 17 selected.

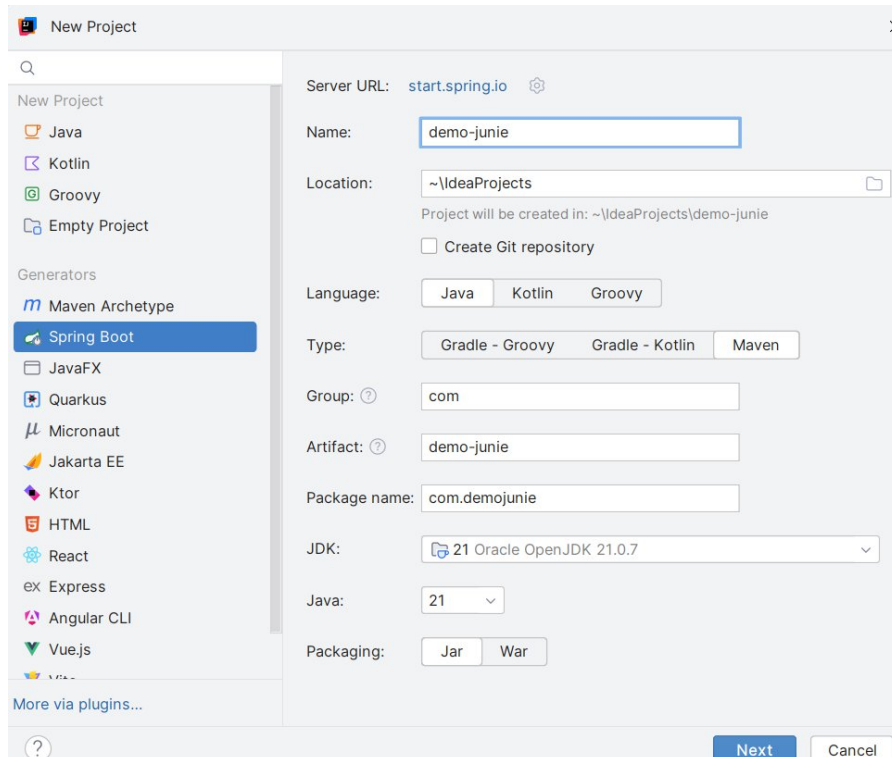
Installer l'assistant IA (Plugin Junie), doc. : <https://www.jetbrains.com/fr-fr/junie/>

Dans IntelliJ : settings → Plugins (ou marketplace pour le chercher) A noter la possibilité de travailler en équipe et directement dans le projet Github.

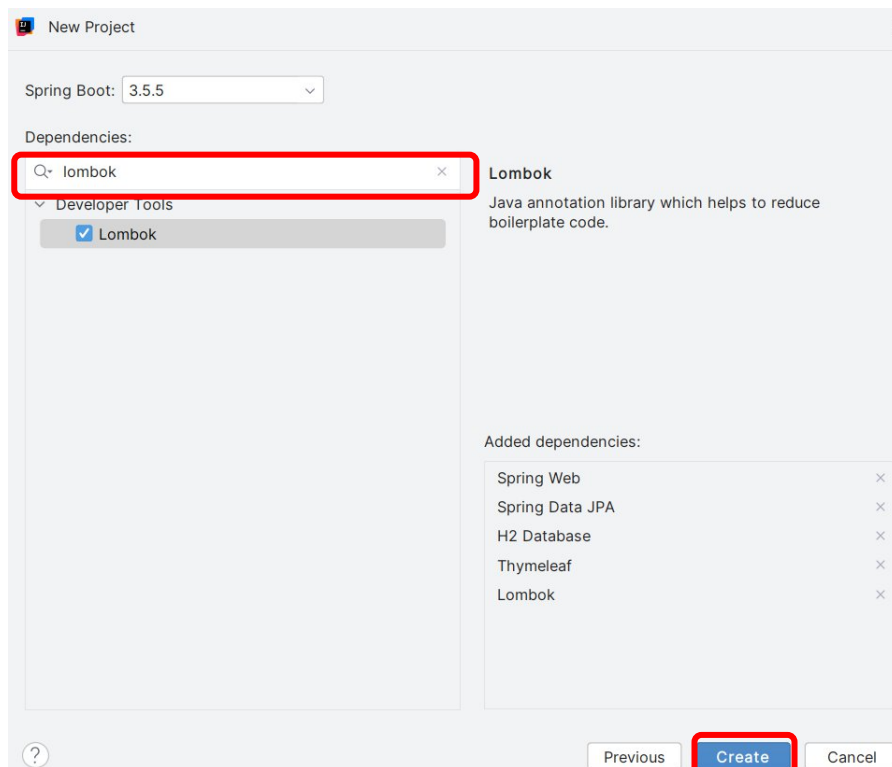


4. Projet demo-junie

Créer un projet Spring Boot :



Ajouter les dépendances suivantes : Spring Web, Spring Data JPA, H2 Database, Thymeleaf, Lombok (uniquement depuis l'onglet recherche), puis Create :



Ouvrez l'assistant IA Junie et entrez le prompt suivant :

```
Créer une application qui permet de gérer des produits (id,
nama, price, quantity) en utilisant Spring Boot, Spring Data
JPA, H2 Database et Thymeleaf
```

Junie corrigera la faute : nama → name

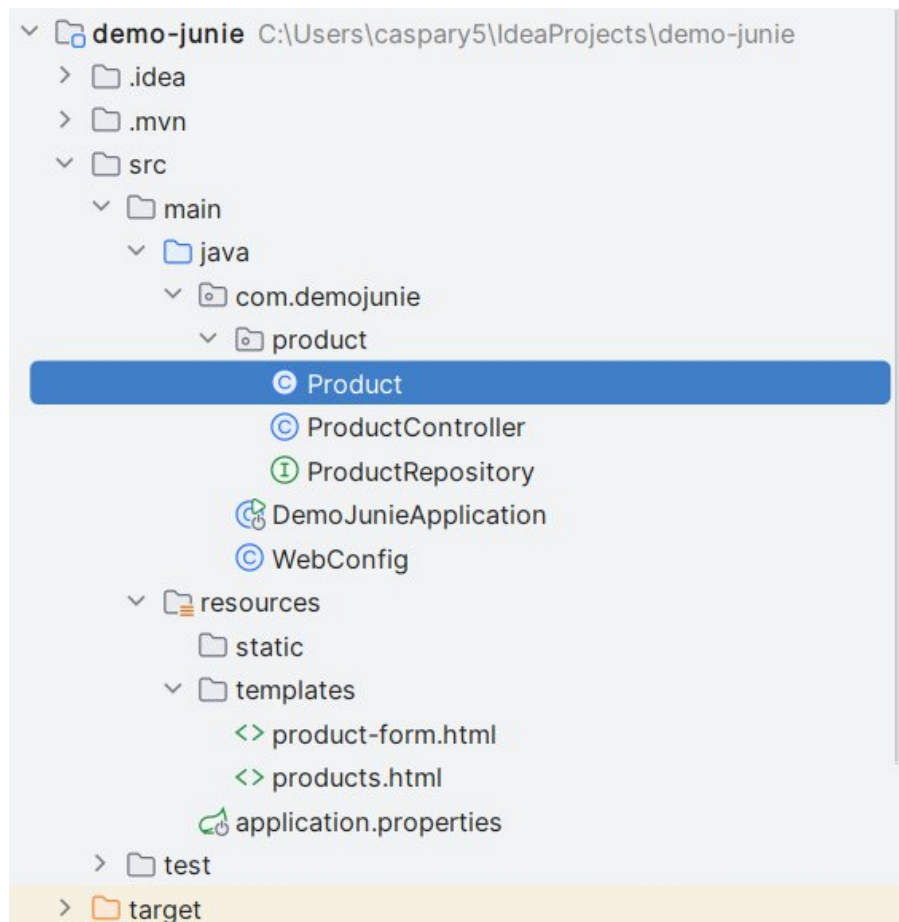
Et va créer les fichiers nécessaires via le LLM... **Approuver** les propositions de Junie.

Puis **lancer (run)** le résultat.

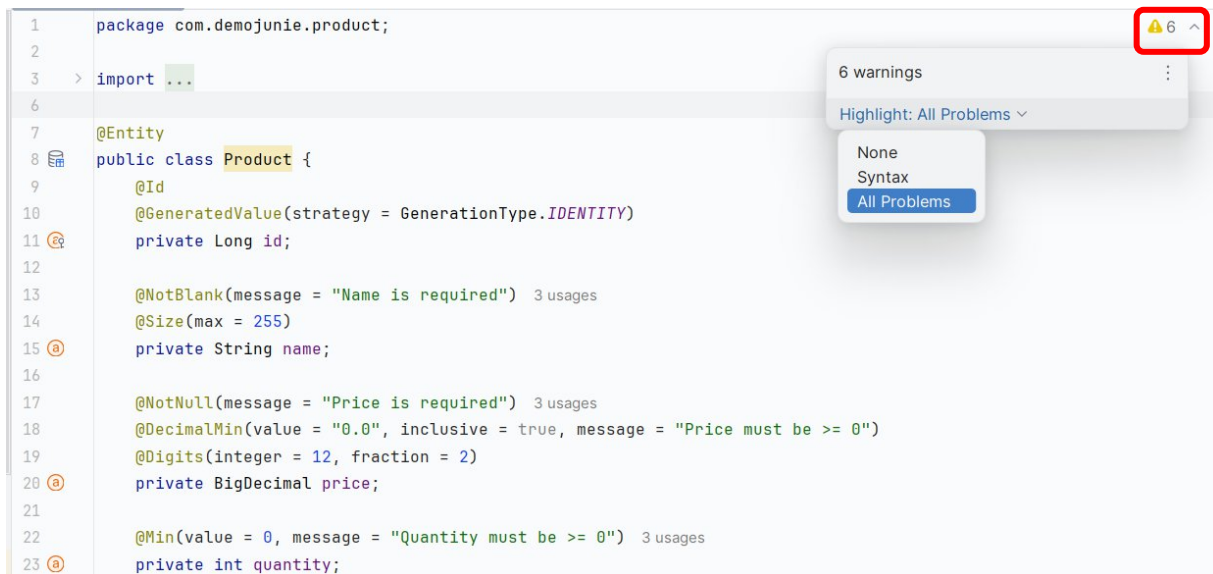
Remarque : les personnes qui ne connaissent pas Spring Boot auront du mal à corriger et à améliorer le code, et plus encore à le compléter « à la main ». L'assistant contribue à améliorer la productivité du développeur mais celui-ci doit acquérir les connaissances nécessaires à la compréhension du code.

Junie vous donne le récapitulatif des fichiers et des lignes de code générés.

La structure du projet est maintenant la suivante :



Ouvrez la classe Product :



```
1 package com.demojunie.product;
2
3 > import ...
4
5
6
7 @Entity
8 public class Product {
9     @Id
10    @GeneratedValue(strategy = GenerationType.IDENTITY)
11    private Long id;
12
13    @NotBlank(message = "Name is required") 3 usages
14    @Size(max = 255)
15    private String name;
16
17    @NotNull(message = "Price is required") 3 usages
18    @DecimalMin(value = "0.0", inclusive = true, message = "Price must be >= 0")
19    @Digits(integer = 12, fraction = 2)
20    private BigDecimal price;
21
22    @Min(value = 0, message = "Quantity must be >= 0") 3 usages
23    private int quantity;
```

6 warnings

Highlight: All Problems

None
Syntax
All Problems

Il y a des warnings signalant notamment que vous pouvez utiliser lombok :



Problems File 6 Project Errors Vulnerable Dependencies AI Self-Review Beta Qodana Security Analysis x

Product.java C:\Users\caspar5\IdeaProjects\demo-junie\src\main\java\com\demojunie\product 6 problems

- Class 'Product' may use Lombok @Getter :8
- Class 'Product' may use Lombok @Setter :8
- Redundant default parameter value assignment :18
- Method 'setName(java.lang.String)' is never used :37
- Method 'setPrice(java.math.BigDecimal)' is never used :40
- Method 'setQuantity(int)' is never used :43

Travail à faire :

- Supprimer tous les warnings.
- Comprendre la structure du code et son fonctionnement
- Lancer l'application : DemoJunieApplication.java

Résultat :



```
1 package com.demojunie.product;
2
3 > import ...
4
5
6
7 @Setter
8 @Getter
9 @Entity
10 public class Product {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @NotBlank(message = "Name is required")
16     @Size(max = 255)
17     private String name;
18
19     @NotNull(message = "Price is required")
20     @DecimalMin(value = "0.0", message = "Price must be >= 0")
21     @Digits(integer = 12, fraction = 2)
22     private BigDecimal price;
```


- **Vérifiez les fonctionnalités de l'application**, dans le navigateur tapez :
localhost:8080/products

Exemple de formulaire via Thymeleaf :

Produits

Nouveau

Rechercher par nom

Rechercher

ID	Nom	Prix	Quantité	Actions	
1	Clavier	29,99	100	Modifier	Supprimer
2	Souris	19,50	200	Modifier	Supprimer
3	Écran	199,00	50	Modifier	Supprimer

- **Changer le port 8080 par défaut en 8084** (où s'effectue le changement ?)
- **Génère un fichier README**

New Files

README.md +87

A comprehensive README.md file was successfully created for the Spring Boot product management app, detailing features, tech stack, usage instructions, and troubleshooting tips, all in French. The README accurately reflects the project's configuration, including the change of the port to 8084, with no build errors reported.

- **Comment obtenir l'interface console de H2 Database et visualiser la table des produits ?** (récupérer l'URL JDBC)

Par défaut :

English
Preferences
Tools
Help

Login

Saved Settings: Generic H2 (Embedded)

Setting Name: Generic H2 (Embedded) Save Remove

Driver Class: org.h2.Driver

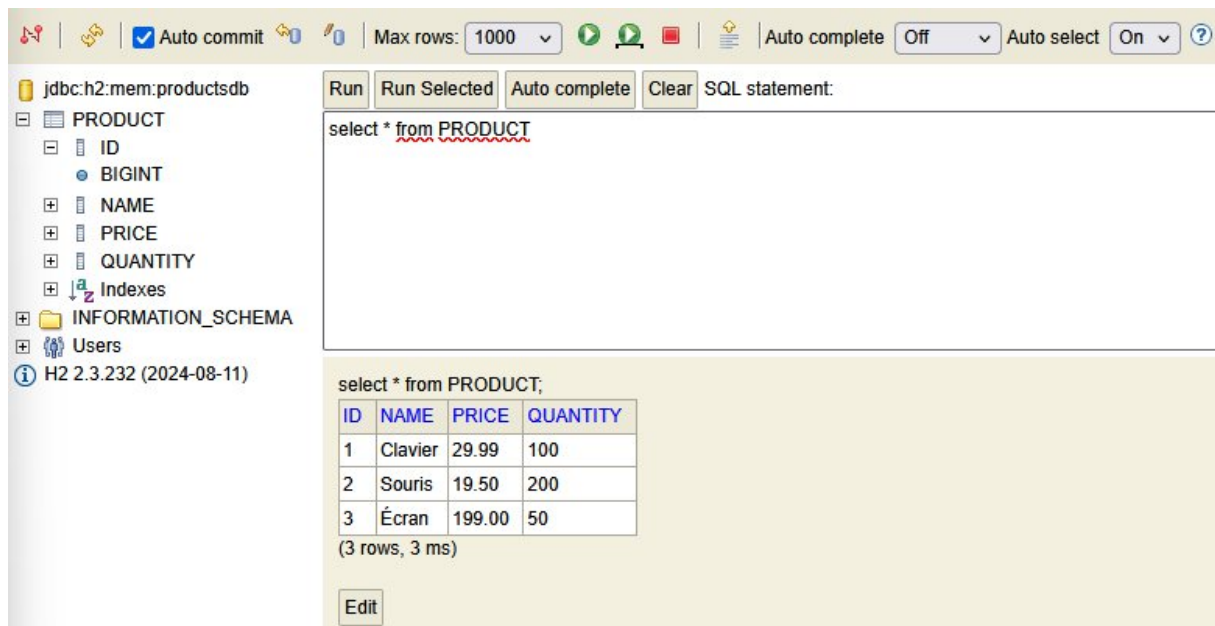
JDBC URL: jdbc:h2:~/test

User Name: sa

Password:

Connect Test Connection

Ce que l'on doit obtenir :



- Insérez un nouveau produit depuis la console H2 (par exemple Ordi)
- Vérifiez la mise à jour de la BDD dans l’affichage du formulaire HTML.

Produits

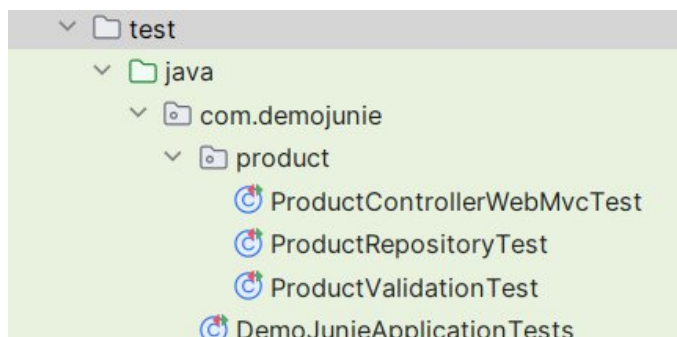
Nouveau

Rechercher par nom

Rechercher

ID	Nom	Prix	Quantité	Actions	
1	Clavier	29,99	100	Modifier	Supprimer
2	Souris	19,50	200	Modifier	Supprimer
3	Écran	199,00	50	Modifier	Supprimer
5	Ordi	2000,00	4	Modifier	Supprimer

- Génère les tests unitaires pour cette application



Corriger les Warnings ou erreurs éventuelles de manière à valider chacun des fichiers de tests.

- Lancer chaque fichier du répertoire test pour les vérifier (tests passed)

Une erreur peut se produire dans la classe ProductRepositoryTest.

Type d'erreur rencontré (et tout à fait possible) :

Le problème vient du fait que l'annotation n'exclut pas le CommandLineRunner défini dans votre classe . Ce CommandLineRunner charge des données initiales dans la base de données, incluant un produit nommé "Souris".

Voici ce qui se passe :

1. Lors du démarrage du contexte de test, le s'exécute et ajoute 3 produits :
CommandLineRunner
 - "Clavier"
 - "Souris" ← Voici le problème !
 - "Écran"
2. Dans votre test , vous ajoutez : `findByNameContainingIgnoreCase_shouldWork()`
 - "Souris Gaming"
 - "Tapis de souris"
 - "Clavier"
3. Quand vous recherchez "SOURIS", vous obtenez 3 résultats au lieu de 2 :
 - "Souris" (du CommandLineRunner)
 - "Souris Gaming" (de votre test)
 - "Tapis de souris" (de votre test)

Solution :

Ajout de la méthode `@BeforeEach setUp()` qui exécute `repository.deleteAll()` garantit que chaque test commence avec une base de données vide, éliminant ainsi l'interférence des données du CommandLineRunner.

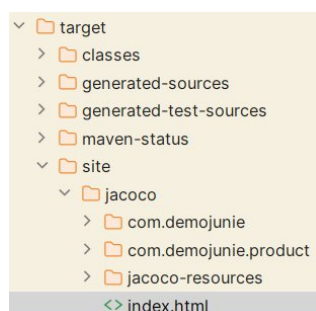
Vous pouvez utiliser Ai Chat lié à un modèle LLM (tel que Claude 4) pour modifier le code et ainsi éliminer le problème.

Icône pour AI Chat :



— Avec Junie : Quelle est la couverture du code ? Vérifie.

L'assistant IA va ajouter le Plugin Jacoco au projet Maven et exécuter les tests via les commandes maven. Le résultat est placé dans un fichier : `target` → `site` → **index.html**



2 PLAN

- 3 ✓ 1. Vérifier la configuration actuelle du projet et l'absence d'outil de couverture.
- 4 ✓ 2. Ajouter JaCoCo au build Maven pour mesurer la couverture et générer les rapports HTML.
- 5 • 3. Exécuter les tests via Maven pour produire les rapports de couverture:
- 6 - mvnw.cmd -q test (Windows) ou ./mvnw -q test (Linux/macOS).
- 7 - Résultats attendus: target/site/jacoco/index.html (rapport HTML), target/jacoco.exec (données).
4. Si l'environnement local bloque le téléchargement des deps de test (cas observé pour @DataJpaTest), documenter comment résoudre (proxy/miroir Maven) et fournir la commande pour lire le rapport.
5. Communiquer le chemin des rapports et une estimation qualitative: les tests couvrent l'entité Product, le repository, et le contrôleur; la couche vue Thymeleaf n'est pas instrumentée directement mais traversée dans les tests MVC.

– Ouvrir le fichier index.html avec un navigateur

Résultat :

demo-junie

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.demojunie.product		14 %		0 %	11 14	29 38	7 10	0 2
com.demojunie		91 %		50 %	2 7	2 12	1 6	0 2
Total	132 of 210	37 %	9 of 10	10 %	13 21	31 50	8 16	0 4

– Avec Junie : Ajoute les tests d'intégration et un rapport.

- ✓ 1. Inspecter les tests existants et cibler les manques (tests d'intégration bout-en-bout).
- ✓ 2. Ajouter des tests d'intégration Spring Boot:
 - ✓ - 2.1. ProductIntegrationTest avec @SpringBootTest + @AutoConfigureMockMvc couvrant CRUD via HTTP.
 - ✓ - 2.2. ProductDataSeedIntegrationTest validant les données seed au démarrage.
- ✓ 3. Ajouter/mettre à jour la documentation du rapport:
 - ✓ - 3.1. README mis à jour: exécution des tests d'intégration et accès au rapport JaCoCo.
- ✓ 4. Exécuter les nouveaux tests pour confirmer:
- ✓ 5. Vérifier l'isolation des données (cleanup @BeforeEach et @Transactional):
- ✓ 6. Finaliser: résumer changements et rappeler les commandes Maven pour générer le rapport JaCoCo.

Des tests d'intégration ont été ajoutés à l'application Spring Boot pour gérer les produits, incluant un test CRUD complet via MockMvc et un test de validation des données seed. Les deux nouveaux tests ont réussi, et le README a été mis à jour pour inclure des instructions sur la génération et l'accès au rapport de couverture JaCoCo.

- Ouvrir à nouveau le fichier index.html avec un navigateur

Résultat :

demo-junie												
demo-junie												
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes				
com.demojunie.product		44 %		37 %	7 14	15 38	3 10	0 2				
com.demojunie		91 %		50 %	2 7	2 12	1 6	0 2				
Total	87 of 210	58 %	6 of 10	40 %	9 21	17 50	4 16	0 4				

Plus en détail :

demo-junie > com.demojunie.product > ProductController
--

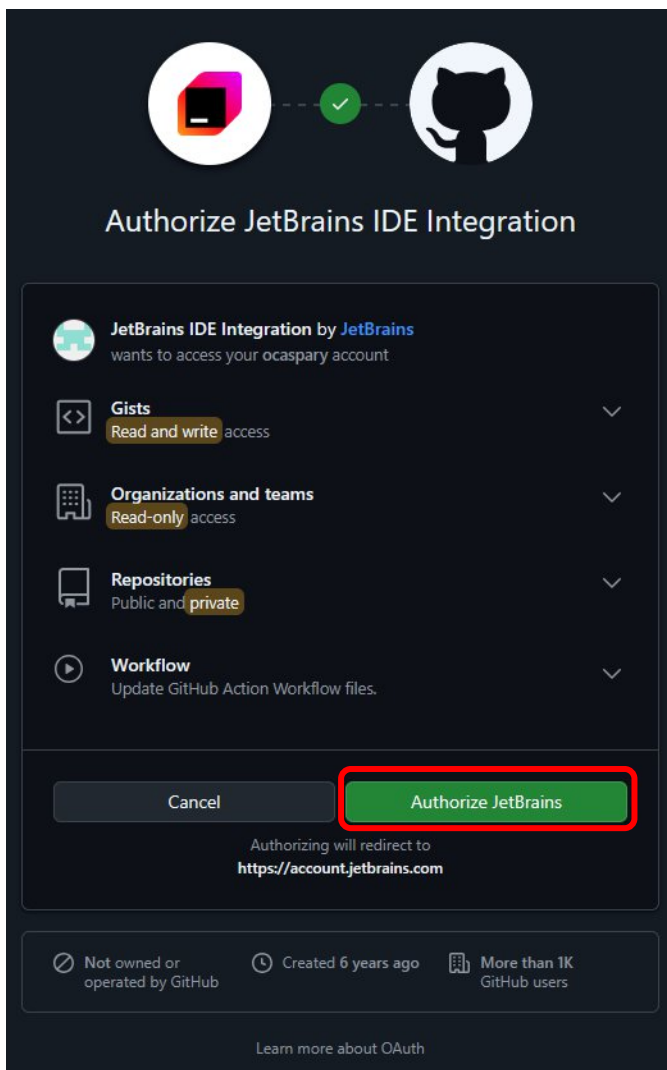
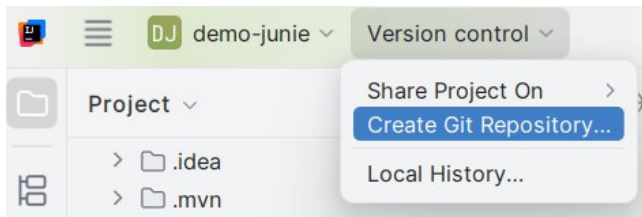
ProductController

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes				
editForm(Long_Model)		0 %			n/a	1 1	5 5	1 1				
createForm(Model)		0 %			n/a	1 1	4 4	1 1				
create(Product_BindingResult_Model)		45 %		50 %	1 2	3 6	0 1	0 1				
update(Product_BindingResult_Model)		45 %		50 %	1 2	3 6	0 1	0 1				
list(String_Model)		73 %		25 %	2 3	0 6	0 1	0 1				
lambda\$editForm\$0(Long)		0 %			n/a	1 1	1 1	1 1				
ProductController(ProductRepository)		100 %			n/a	0 1	0 3	0 1				
delete(Long)		100 %			n/a	0 1	0 2	0 1				
Total	82 of 133	38 %	5 of 8	37 %	7 12	15 32	3 8	3 8				

- Regardez le répertoire test, que constatez-vous ? Apports des nouveaux fichiers ?
- Avec Junie : Faites un dépôt du projet demo-junie sur votre compte github.

- ✓ 1. Préparer le projet pour GitHub.
- ✓ 2. Configurer CI GitHub Actions.
- ✓ 3. Fournir les commandes précises pour créer/pousser le dépôt sur "ocasparty".
 - 4. Optionnel: Renseigner la section <scm> du pom.xml une fois l'URL connue.
- ✓ 5. Vérifications finales.
6. Finaliser: Résumé et pas-à-pas pour publication.

Le projet a été préparé pour un dépôt GitHub avec un .gitignore et un workflow GitHub Actions pour la CI. L'utilisateur doit créer le dépôt "ocasparty/demo-junie" et suivre les commandes fournies pour le pousser. La CI générera automatiquement les artefacts, y compris le rapport JaCoCo.



A noter la création des fichiers .gitignore et ci.yml pour le dépôt (repository) github.

 **Edit** .gitignore +51

The .gitignore file was successfully created to prepare the project for publication on GitHub. It includes specific entries to exclude unnecessary files and directories from version control.

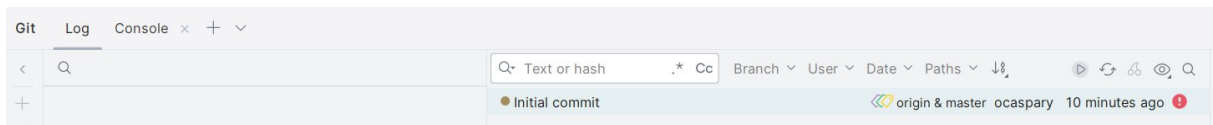
 **Edit** ci.yml +36

A GitHub Actions workflow for continuous integration has been created to handle builds, tests, and JaCoCo report uploads on pushes and pull requests.

 **Edit** README.md +57

The README file has been updated with instructions for creating a GitHub repository and pushing the project. GitHub Actions CI workflow is configured for automatic builds and tests.

Commit initial du projet :



A noter une erreur de droits pour lancer mvn dans le fichier ci.yml.

– Vérifier votre dépôt sur Github

Le projet sur github.com :

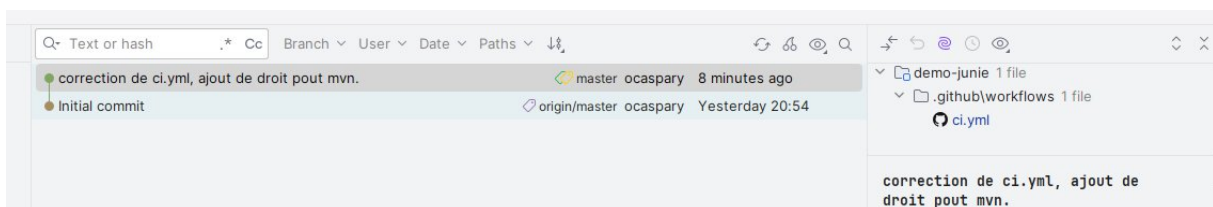


– Avec Junie : corrige le fichier ci.yml car il y a une erreur de permission pour lancer mvn.

- ✓ 1. Inspect the ci.yml to identify where to fix the permission issue.
- ✓ 2. Add a step to grant execute permission to mvnw before the build step (chmod +x mvnw).
- ✓ 3. Keep all other steps unchanged to ensure minimal changes and preserve existing behavior.
- ✓ 4. Explain the change and how it resolves the permission error.
- 5. Finalize and submit the fix.

The GitHub Actions workflow was updated to address a permission error when launching `mvn` by adding a `chmod +x mvnw` step before the build process. The fix was successfully implemented and the changes submitted.

- **Faire un commit pour mettre à jour le fichier ci.yml.**



- **Vérifier la mise à jour sur Github**

Remarque : Configuration CI/CD

Nous n'aborderons pas la partie CI/CD.

Cependant, pour que votre workflow GitHub Actions fonctionne avec Maven (mvn) dans un fichier de workflow CI (.github/workflows/ci.yml par exemple), il faut généralement :

1. Installer Java et Maven dans le workflow à l'aide d'actions appropriées.
2. Définir les permissions si le workflow doit accéder à des secrets, packages, ou artefacts protégés.
3. Utiliser les bonnes permissions dans le bloc permissions: du workflow si vous publiez des packages ou accédez à des ressources GitHub.

Voici un exemple de workflow minimal pour Maven avec permissions (ci.yml) :

```
name: Java CI with Maven
```

```
on:
```

```
  push:
```

```
    branches: [ "main" ]
```

```
  pull_request:
```

```
    branches: [ "main" ]
```



```
permissions:
  contents: read    # nécessaire pour checkout
  packages: write   # si vous publiez sur GitHub Packages
  # id-token: write # si vous utilisez OIDC pour un registre externe

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
      - name: Build with Maven
        run: mvn clean install
```

Prompts (à faire) :

- Pour le plugin lombok en Java, fais moi une liste des annotations et de leur usage.
- Génère un tableau en Word.
- Quelle est l'annotation qui permet l'écriture du code comme en POJO ?
- POJO versus lombok pour les tests unitaires ?

Mise en œuvre de SpringBuilder

Exemple d'une classe POJO (Plain Old Java Object) à la fin de l'article suivant :

<https://medium.com/@Bharat2044/what-is-pojo-class-in-java-1325eb7f3a6c>

- **Travail à faire : Remplacer toString par StringBuilder.**

POJO classique vs Lombok @Data

POJO classique (sans Lombok)	POJO avec Lombok (@Data)
<pre>public class User { private String name; private int age; public User() { } public User(String name, int age) { this.name = name; this.age = age; } public String getName() { return name; } public void setName(String name) { this.name = name; } public int getAge() { return age; } public void setAge(int age) { this.age = age; } @Override public String toString() { return "User{name='" + name + "', age=" + age + "'}"; } @Override public boolean equals(Object o) { if (this == o) return true; if (!(o instanceof User)) return false; User user = (User) o; return age == user.age && name.equals(user.name); } @Override public int hashCode() { return java.util.Objects.hash(name, age); } }</pre>	<pre>import lombok.Data; @Data public class User { private String name; private int age; }</pre>

Annotations Lombok - Référence rapide

Annotation	Usage	Exemple
@Getter	Génère les getters	@Getter private String name;
@Setter	Génère les setters	@Setter private int age;
@ToString	Génère toString()	@ToString(exclude="password") class User { ... }
@EqualsAndHashCode	Génère equals() et hashCode()	@EqualsAndHashCode class User { ... }

@Data	Regroupe Getter, Setter, ToString, EqualsAndHashCode, RequiredArgsConstructor	@Data class Person { private String name; }
@Value	Version immuable de @Data (tous champs final)	@Value class Point { int x, y; }
@NoArgsConstructor	Constructeur vide	@NoArgsConstructor class User { ... }
@RequiredArgsConstructor	Constructeur pour champs final ou @NonNull	@RequiredArgsConstructor class Service { private final Repo repo; }
@AllArgsConstructor	Constructeur avec tous les champs	@AllArgsConstructor class User { String n; int a; }
@Builder	Génère un builder pattern	User.builder().name("Olivier").age(40).build();
@SuperBuilder	Builder compatible avec héritage	class Child extends Parent { @SuperBuilder Child(...) }
@NonNull	Vérifie non-nullité et jette NullPointerException	User(@NonNull String name)
@Slf4j	Génère un logger SLF4J	log.info("Message");
@Log / @Log4j / @Log4j2 / etc.	Logger pour framework choisi	@Log4j2 class App { log.debug("..."); }
@UtilityClass	Classe utilitaire statique (finale + constructeur privé)	@UtilityClass class Utils { void doSomething(){} }
@Cleanup	Ferme automatiquement une ressource (try-with-resources)	@Cleanup InputStream in = ...;
@SneakyThrows	Lancer une exception sans throws	@SneakyThrows void f(){ throw new Exception(); }
@With	Méthodes immuables withXxx()	user.withName("NewName")
@Singular	Dans un builder, gère les collections	@Singular List<String> roles;
@Delegate	Délègue les méthodes à un champ	@Delegate private List<String> list = new ArrayList<>();
@Accessors	Configure getters/setters (fluent, prefix...)	@Accessors(chain=true) @Getter @Setter
@FieldDefaults	Définit des modificateurs par défaut	@FieldDefaults(level=AccessLevel.PRIVATE, makeFinal=true)
@Experimental	Marque une fonctionnalité instable	@Experimental class Test { ... }